Project Report: Breast Cancer Detection

Kushagra Tiwary

Advisors: Minh. Do, Vaishnavi Subramanian

December 23, 2017

Abstract

We propose an end-to-end pipeline to detect Breast Cancer tumorous regions on Whole Slide Images by detecting for possible red flag features using deep learning. We do so by first extracting features that we deem relevant to a higher probability of Breast Cancer, like Lymphocyte and Nuclei patterns, and then analyze the spatial features of those binary maps to predict if a proposed region is tumorous or benign.

1 Introduction

Over the summer we had developed a Convolutional Neural Network for detection of lymphocytes and an auto encoder for segmenting nuclei as proposed by Janowczyk et al. The network achieved the roughly the same accuracy as proposed by Janowczyk et al. However, as we started to test the network on complex images, two problems became very evident: the network was slow in detection of larger images, and couldn't handle complex datasets of whole slide Images like TCGA which has many features such as lymphocytes, nuclei, tissues etc. In particular, the network confuses between lymphocytes and nuclei often, marking both or neither of them. This semester Vaishnavi and I had set the following goals for me. I will explain each of those goals in the report.

Finishing up the Current Network:

1. Implement Non- Maximum Suppression so that the output maps can be cleaned up
2. Converting the network into fully convolutional for easy processing of larger images
3. Running the faster network on TCGA Dataset

Improving the Performance of the Network:

1. Incorporate Nuclei Images as negative patches for lymphocyte detection for greater accuracy on complex datasets
2. Converting the network into a multi-class network by incorporating nuclei images, and tissues as another class

2 Finishing up the Current Network

2.1 Current Network Architecture

The current network is a version of the AlexNet Architecture with pre-trained weights taken from Janowczyk et al. and is used to detect lymphocytes. The network uses a sliding window approach to detect for lymphocytes throughout the larger image. If a lymphocyte is detected within that window, a mark is placed in the center which pertains to the predicted center of the lymphocyte by the model.

The major difference in between ours and their network is that our network stain normalizes the input images first. This is done so that color doesn't play a major factor when we process more complex images from the TCGA or Cameylon16 Dataset. To account for this change in input images, we had to retrain the network. The results are displayed in the table below:

	Positive	Negative
True	701	N/A
False	36	269

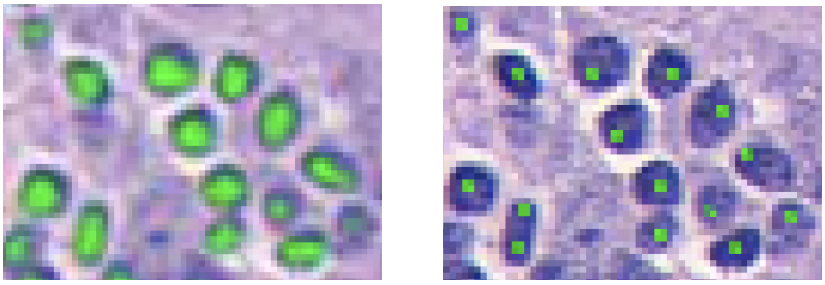
Table 1: Confusion Matrix over 35 random Images

One of the things that is immediately evident is that the False Negative count is very high. To investigate this, I went over all of the 35 images and found out that over 40% of the false negatives are due to lymphocytes near the edge of the image. Since the sliding window is of size 32 by 32, in reality only an image of size *[15, H-15]* by *[15,W-15]* is processed. This makes these edge lymphocytes undetectable by the network.

We calculate the *f-score* to be 0.822, TPR to be 0.722, and the PPV to be 0.951. The accuracy is 82.34%. However, the *f-score* over random 35 images over 10 iterations was 0.841 which shows that there are a group of images on which the network isn't performing so well. This is slightly lower than Janowczyk's network's *f-score* of 0.90, however, his data was taken over randomly selected 21 images. Decreasing the false negative count is one of the issues I look at with the new proposed network as well and have explained a method which can help decrease the **FN** count.

2.2 Non-Maximum Suppression

One of the problems in object detection is that the algorithm might find multiple detections of the same object. The current models works by sliding a window of size 32 by 32 over an input image, and then marking the four center pixels if a lymphocyte was detected. However, the sliding window with a stride of 6 pixels detected the same lymphocyte multiple times- for an image with roughly 32 lymphocytes, the network detected 317 patches therefore clearly detecting each lymphocyte multiple times. Figure 1 shows a sample output of what happens if each lymphocyte is detected multiple times. This masking effect is undesirable because we want to analyze the spatial patterns of these lymphocytes.



(a) Without NMS (b) With NMS

Figure 1: Sample outputs of the network with and without NMS

We use Non-Maximum Suppression to suppress multiple detections of the same object. NMS greedily chooses the best bounding box from a given list by analyzing its "score" and the Intersection over Union (IOU) ratio of other boxes. If a sliding window classified a patch as positive, then that given box was added to the list. The "score" of a bounding box was its probability which tells the algorithm to choose the box that we are most confident with and calculate the IOU ratios based on that box. If the IOU of one box is greater than a particular threshold then that box ix suppressed i.e removed from the list. After running the above procedure iteratively, we mark the center pixels of each of those bounding boxes as shown in Figure 2.b.

In Figure 2.b, one of the bounding boxes shows that there are 2 lymphocyte centers marked on a single lymphocyte. This is because the lymphocyte is long, and therefore, the Intersection over Union ratios are smaller for boxes at either end which leads the algorithm to think that there are two lymphocytes instead of one. In future experiments I have tried to tighten the threshold condition for detection for the NMS algorithm, however, not enough changes were detected. This is a rare case, however.

2.3 Fully Convolutional Network

The other major problem we faced was the time it took to run inference on larger images. The network uses a stride based approach in classifying the whole image. Particularly, the network's input size was fixed to 32 by 32, and we then slided that window over larger images to detect for lymphocytes. As suggested by Raymond during the presentation this is highly inefficient. Moreover, the paper uses a larger window size that necessary to detect lymphocytes, and then accounts for it by up sampling the images to 80x magnification. Since most TCGA images are 40x, we have to up sample each image, essentially doubling the processing time due to a flawed network design.

Moreover, since we want the output binary map size to be equal to the input size, we have to down sample an image afterwards. This extra computation per image is an additional cost because of the inherent design of the network. In fact, Vaishnavi and Raymond showed after the presentation how using a window size of 32 versus 16 doesn't have any tangible effect on the inference. This leads me believe that the researchers re-used this network because all the other cases in the paper use the same model. However, since this pre-trained weights exist of a network of this architecture, the network is converted to a fully convolutional network to reduce the inference cost.

To convert the existing network into fully convolutional I first looked at the Fully Convolutional Networks for Semantic Segmentation by Long et al. The paper proposes to convert the fully connected layers into fully convolutional. In our case that means that we can reuse the last layers weights, change their shape and use them as a convolutional kernel instead. We make sure that an input size of [32,32,3] outputs a [1,1,2] vector- one element for each binary output for is Lymphocyte and is Not Lymphocyte. After the conversion of Fully Connected to Fully Convolution, our network now has the following architecture:

1. Variable Input Size
2. Convolution Layer without Relu of Filter Size [5,5,3,32]
3. Max Pooling Layer of Kernel size [1,3,3,1] and stride 2
4. Convolution Layer without Relu of Filter Size [5,5,3,32]
5. Activation Layer Relu
6. Convolution Layer with Relu of Filter Size [5,5,32,32]
7. Average Pooling Layer of Kernel size [1,3,3,1] and stride 2
8. Convolution Layer with Relu of Filter Size [5,5,32,64]
9. Average Pooling Layer of Kernel size [1,3,3,1] and stride 2
10. Convolution Layer with Relu of Filter Size [4,4,64,64] (Previously FC)
11. Convolution Layer with Relu of Filter Size [1,1,64,2] (Previously FC)

These filter sizes makes sure that for an input size of 32 by 32 we get a binary classification, therefore emulating the older network.

2.3.1 Fully Convolutional for Larger Image Sizes

A Fully Convolutional Layer can be used to provide Pixel-Wise prediction by up-sampling the output layer using deconvolution layers leading to a segmented output with marked lymphocyte centers. [2] However, due to a heavy use of pooling layers by the network, our convolution output size is different than the input size, therefore we are not able to use such a scheme. For example, for an input size of 400 by 400 by 3, the output size is of size 47 by 47 by 2 as opposed to 376 by 376 by 2.

Upon closer inspection with Vaishnavi we were able to realize that this is due to pooling layers in the network. Each of the pooling layers pools with a kernel size of 3 and a stride of 2 hence

decreasing the height and the width by 2 at each step. This means that for each 3 by 3 window $w_{0,0}$ there are 3 additional windows $w_{0,2}$, $w_{2,0}$ and $w_{2,2}$, where the original values of $w_{0,0}$ overlap.[1]

This means that we will need to run the rest of the convolutions on each of those shifted windows as well. This is done that the every possible output after the strided pooling layer are accounted for in the final segmented image. As shown in Figure 2, for a kernel size of 3 by 3, and stride of 2, the values x_3, x_6, x_7, x_8, x_9 are shared. Therefore for each of those *orange* $w_{0,0}$, *green* $w_{2,0}$, *yellow* $w_{0,2}$ and *blue* $w_{2,2}$ windows need to be combined based on their offsets from their respective base windows, and then those feature maps needs to be convolved separately by the whole network.

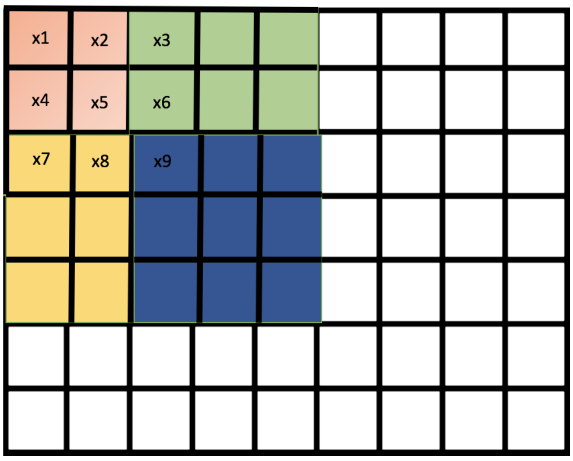


Figure 2: Additional windows using the same values created by using pooling of kernel size 3, and stride 2 which needs to be taken into account

At the last convolution layer these, individual outputs will then need to be combined so that output of the network is a segmented image.

2.3.2 Revised Network Architecture

As explained above, the strided pooling in the network makes it difficult to convert the architecture to fully convolutional. In fact, the presence of three different pooling makes the new architecture significantly more complex.

The first convolution layer followed by max pooling causes the rest of the network to process 4 different outputs. Another average pooling layer after the second convolution layer causes each of those 4 earlier outputs to spawn additional 4. The third convolution layer with average pooling causes additional 4 outputs for each previous output to be processed by the rest of the network. Therefore, following this routine, the network will now have 64 outputs instead of one. These outputs will need to be assembled in a manner which is discussed later in the paper. As shown in figure 3, for an input size of 400 by 400 by 3 the revised network will now have a correct output size of 376 by 376 by 2 after combining each of the network's outputs. The reason why we should get an output of size 376 instead of 400 is because the convolution kernel doesn't process lymphocytes at either of the edges in the image.

[1]The subscript refers to a shift *(x,y)* in the original window $w_{x0,y0}$ where $w_{x0,y0}$ is a 3 by 3 window with *(x0,y0)* as its top left coordinate.

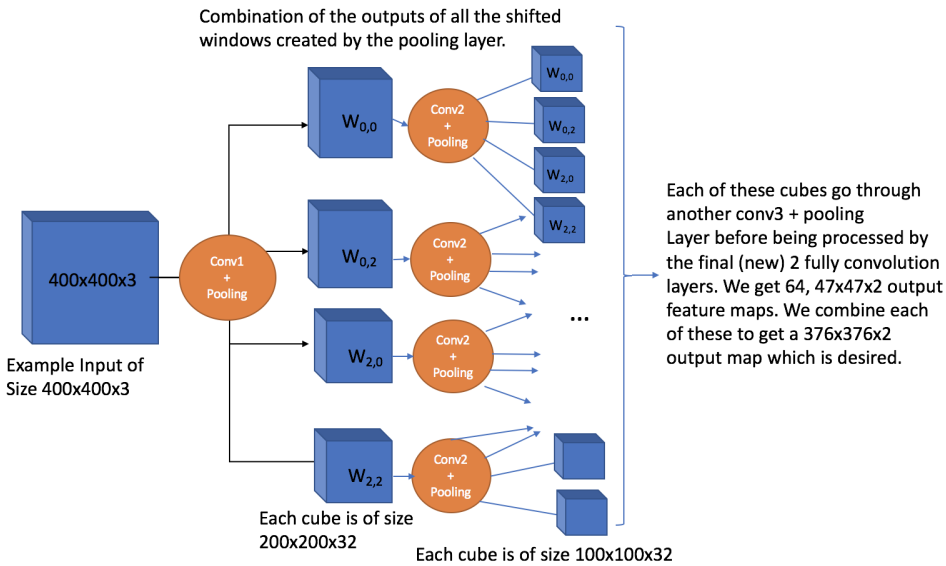


Figure 3: The Revised Network Architecture after accounting for strided pooling

2.3.3 Implementation of the Revised Fully Convolutional Network

There are two phases to implement the revised network. The first phase involves training the network. This phase is pretty straight forward: we load the weights of the convolutional layers and the fully connected layers and then train the network for a few epochs to achieve a 81.78% accuracy. This gives us a trained network however, one that is isn't as efficient as a FCN should be since it still needs to slide for proper size outputs.

To implement the second phase of FCN, we need to implement this using k-d trees. The revised network can be technically looked as a quad tree: each node as a convolution plus pooling operation, and each branch as a new output the network has to now process. Each level of the quad tree is one convolution operation since the same operation will be applied to each of the inputs at that level. This will make combining the pixels at the last level much easier. Thinking about how each of the pixels will combine after 3 pooling layers is a little complex, and Vaishnavi has been helping me to figure out how we can even do this. The combination is very systematic; the parent node's first branch pixels goes in first, followed by the second branch and so on, recursively going up the tree to process a different parent's children. After processing all the leaves, we would have created the required output image.

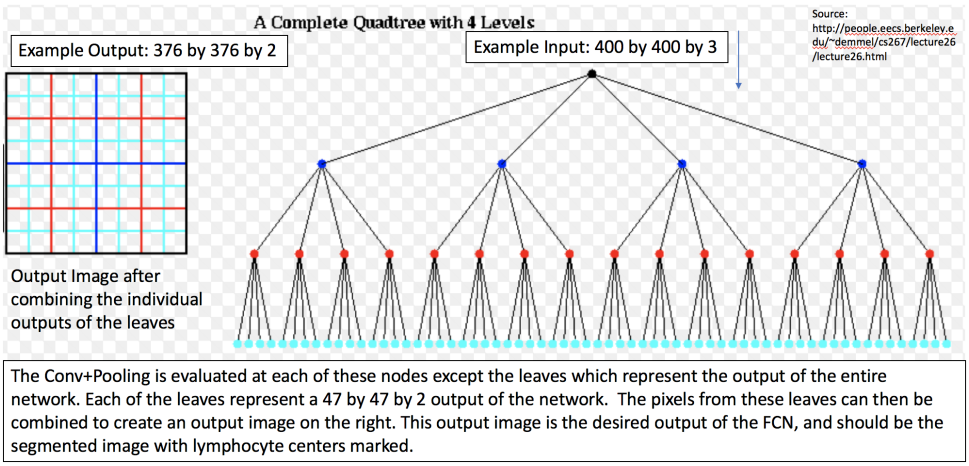


Figure 4: Using a Quad-Tree to map individual outputs of the network to create the segmented output for lymphocyte detection

This sort of implementation is good because what we are doing in the network is convolving with all possible strided outputs, hence we are creating the output image as a combination of these smaller outputs. This problem is very similar to describing an image using quad trees which breaks down an image to each pixel at the leaves. Instead of individual pixels at the leaves, we now have one part of the possible segmented output of the network.

I am still implementing the last part of combining the individual features to create a segmented output, and then will test the newly trained FCN for its *f-score* after the winter break. Running some initial tests on TCGA outputs and comparing them with outputs of the Non-FCN has shown similar results.

3 Improving the Performance of the Network

Going forward there are a couple of ways to improve the network. To improve the *f-score*, and lower down the false negative rate, we need to add negative patches in the form of nuclei to the training data so that the network knows how to distinguish between nuclei and lymphocytes- something that becomes a problem when we run inference on TCGA images. I intend to use patches from Wei-Zhao's research project. Since this is a small network, training will roughly take 12-14 hours.

Nevertheless, a better alternative would be to extend this to a multi-class network which detects for both nuclei and lymphocytes. This might be a better approach than using two separate models for detection, however, we first want to get some important insight on the spatial patterns of these lymphocytes before expanding the project, therefore this goal was pushed away in lieu of getting to the spatial analysis part.

After the full implementation of the FCN, we will run this on the TCGA images at a much faster rate since currently it takes a gruesome 20 minutes to run on a TCGA Image of size 2200 by 2200, and whole slide images are much larger. Going forward, now that I have good outputs, I will study spatial features: Wei-Zhao has suggested to look at the K function and clustering as a good place to start.

References

[1] Janowczyk, A. and Madabhushi, A. *Deep learning for digital pathology image analysis: a comprehensive tutorial with selected use cases.* J. Pathol. Inform. 7, 29 (2016).

[2] J. Long, E. Shelhamer, and T. Darrell. *Fully convolutional networks for semantic segmentation.* arXiv:1411.4038, 2014.